

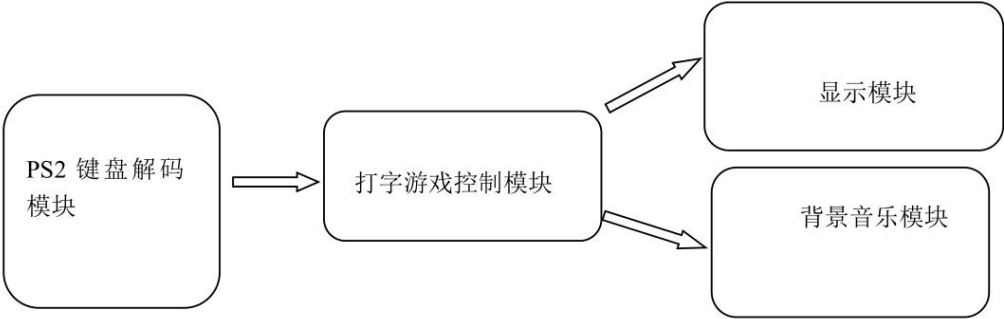
PS/2 键盘打字游戏

1 、实验目的:

基于 FPGA 技术实现打字游戏的一种方法。VGA 现实和 PS2 键盘输入两个模块，来实现打字游戏。

2 、实验原理:

打字游戏的原理框图如图 2.1 所示。它主要有 PS2 键盘解码模块、打字游戏控制模块、显示模块以及背景音乐模块。



2.1 打字游戏的设计框图

打字游戏控制主要是基于点阵思想，在程序上通过状态机实现。显示器的显示可以看出由列与行为坐标构成的点阵，可以通过控制某一特定的位置来进行显示。

通过外围的模可控制计数器，来产生随机字符，并通过 VGA 显示在屏幕上。通过状态机是的随机书出现如弹球形式的运动状态。

通过 PS2 键盘来进行数字的输入，通过 PS2 的电路得到输入的数据，并用输入的数据与随机数产生的数据进行比较如果两个数据相等，则将 aa 置 1，公内部处理，并输出。

如果判断正确，会产生如下的效果：爬动的动态阿狸图案会变色，显示器的下方的进度条会向前增加一格，如果没有打对，进度条会保持不变。

最后如果通过了游戏，即进度条到达的边界，会产生动态的蜡笔小新的图案，表示庆祝。

乐曲由音调和音长组成，只要控制好音调和音长并且准确的驱动蜂鸣器就能演奏出动听的乐曲。

该模块通过蜂鸣器产生不同的音乐，并且通过数码管实时的显示音乐音高。其模块框图如图 2.2 所示。



图 2.2

根据简谱音符的音高控制蜂鸣器频率。

数字电路中，模 N 的计数器溢出信号的频率是计数脉冲信号频率的 $1/N$ 。所以，可以设计一个模可控的计数器实现不同的分频比，使溢出信号频率满足不同的音高要求。其音高对应的分频比如图 2.3 所示。由于三个 8 度音程中相同音名的信号频率相差一倍。

所以在分频系数不变时，将计数脉冲频率升高（或降低）一倍可使分频器的溢出信号频率升高（或降低）一倍，音调升高（或降低）8 度。

音名	分频比	预置数	音名	分频比	预置数	音名	分频比	预置数
低音 1	11468	4916	中音 1	5736	10647	高音 1	2867	13516
低音 2	10222	6168	中音 2	5111	11272	高音 2	2555	13829
低音 3	9102	7281	中音 3	4552	11831	高音 3	2276	14108
低音 4	8592	7791	中音 4	4296	12087	高音 4	2148	11535
低音 5	7653	8730	中音 5	3827	12556	高音 5	1913	14470
低音 6	6818	9565	中音 6	3409	12974	高音 6	1704	14678
低音 7	6073	10310	中音 7	3036	13346	高音 7	1518	14864

图 2.3 音高对应分频比

音长控制：某个音符音长由该音高频率信号的持续时间长短决定，可采用一个计数器对音长进行定时，计数脉冲周期是乐谱时值，决定了该单元音符的持续时间。

所以，可采用计数器对音长进行定时，计数脉冲周期为所选乐谱中时值最短音符的时值，其它音符的音长定时可根据其时值与最短音符音长（度量单位）的倍数关系持续不同的计数脉冲周期。比如：乐谱中最短音符为 8 分音长，则 4 分音符为 2 个时钟周期，而全音符为 8 个时钟周期。

3、实验程序：

控制模块程序:

```
module
ball3(clk_25m, r, g, b, hs, vs, clk_count, rst, ps2_clk, ps2_dat, seed, q, DOUT,
DOUT1, rgin, aa);
    input[5:0] seed;//////////
    input clk_25m;
    input clk_count;
    input rst;
    input ps2_clk;
    input ps2_dat;
    input [7:0] q;//////////
    input [199:0] rgin;
    output [9:0] DOUT;//////////
    output r, g, b, hs, vs, aa;
    reg [9:0] x_cnt;
    reg [9:0] y_cnt;
    reg [9:0] x_ball;
    reg [9:0] y_ball;
    reg [2:0] rgb;
    reg hs, vs;
    reg [2:0] state;
    reg flag_x;
    reg flag_y;
    reg [9:0] x_block;
    reg [7:0] ps2_byte_buf;
    reg ps2_clk_r0;
    reg ps2_clk_r1;
    reg [3:0] cnt;
    reg[9:0] DOUT;
    reg a;
    reg aa;
    reg bb;
    reg[199:0] charbit;
    output [10:0] DOUT1;
```

```

reg [10:0] DOUT1;
reg [5:0] data; ///////////////
reg[5:0] data1;
reg[4:0] time1; ///////////////
assign r=rgb[2];
assign g=rgb[1];
assign b=rgb[0];
always @(posedge clk_25m )
if(rst)
begin
if(x_cnt==10'd800) begin x_cnt<=10'd0;end
else x_cnt<=x_cnt+1'b1;
end
always @(posedge clk_25m)
if(rst)
begin
if(x_cnt==800)
begin
if(y_cnt<10'd525) y_cnt<=y_cnt+1'b1;
else y_cnt<=10'd0;
end
end
always @(posedge clk_25m)
if(rst)
begin
if((x_cnt>=640+8+8)&&(x_cnt<640+8+8+96)) hs<=1'b0;
else hs<=1;
end
always @ (posedge clk_25m )
if(rst)
begin
ps2_clk_r0<=ps2_clk;
ps2_clk_r1<=ps2_clk_r0;
end

```

```

wire ps2_clk_n=(~ps2_clk_r0)&ps2_clk_r1;
always @(posedge clk_25m )
if(rst)
begin
if(ps2_clk_n)
begin
if(cnt==4'hA ) cnt<=0;
else cnt<=cnt+1'b1;
end
end
always @ (posedge clk_25m )
begin
if(ps2_clk_n)
begin
case(cnt)
4'h1:ps2_byte_buf[0]<=ps2_dat;
4'h2:ps2_byte_buf[1]<=ps2_dat;
4'h3:ps2_byte_buf[2]<=ps2_dat;
4'h4:ps2_byte_buf[3]<=ps2_dat;
4'h5:ps2_byte_buf[4]<=ps2_dat;
4'h6:ps2_byte_buf[5]<=ps2_dat;
4'h7:ps2_byte_buf[6]<=ps2_dat;
4'h8:ps2_byte_buf[7]<=ps2_dat;
endcase
end
end
always @(y_cnt)
if(rst)
begin
if((y_cnt>=480+10)&&(y_cnt<480+12)) vs<=1'b0;
else vs<=1'b1;
end
always @ (posedge clk_25m) if(rst)
begin

```

```
case(ps2_byte_buf)
8' h1c:data1<=0;
8' h32:data1<=1;
8' h21:data1<=2;
8' h23:data1<=3;
8' h24:data1<=4;
8' h2b:data1<=5;
8' h34:data1<=6;
8' h33:data1<=7;
8' h43:data1<=8;
8' h3b:data1<=9;
8' h42:data1<=10;
8' h4b:data1<=11;
8' h3a:data1<=12;
8' h31:data1<=13;
8' h44:data1<=14;
8' h4d:data1<=15;
8' h15:data1<=16;
8' h2d:data1<=17;
8' h1b:data1<=18;
8' h2c:data1<=19;
8' h3c:data1<=20;
8' h2a:data1<=21;
8' h1d:data1<=22;
8' h22:data1<=23;
8' h35:data1<=24;
8' h1a:data1<=25;
8' h45:data1<=26;
8' h16:data1<=27;
8' h1e:data1<=28;
8' h26:data1<=29;
8' h25:data1<=30;
8' h2e:data1<=31;
8' h36:data1<=32;
```

```

8' h3d:data1<=33;
8' h3e:data1<=34;
8' h46:data1<=35;
8' h4e:data1<=36;
8' h55:data1<=37;
8' h5d:data1<=38;
8' h54:data1<=39;
8' h58:data1<=40;
8' h4c:data1<=41;
8' h4a:data1<=42;
endcase
end

/*always @(data1)
if(data1==data) */
always @ (posedge clk_25m)
if(rst)
begin
if((aa==1)&&(x_cnt==0))begin    data<=seed;x_block<=x_block+20;begin
if(x_block==400)
begin x_block<=400;end end end
else begin data<=data;x_block<=x_block;end
end
else
x_block<=0;
always @(posedge clk_25m)
if(rst)
if((x_cnt==0)&&(y_cnt==100))begin    time1<=time1+1;if(time1==60)
time1=0;end
always @(posedge clk_25m)
if(rst)
begin
if(((x_cnt>=640)&&(x_cnt<800)) || ((y_cnt>=480)&&(y_cnt<525)    ))
rgb<=3'b000;
else

```

```

        if(((x_cnt<10)&&(x_cnt>=0)&&(y_cnt>=0)&&(y_cnt<460)) || ((x_cnt<420)&&
(x_cnt>=0)&&(y
        _cnt>=0)&&(y_cnt<10)) || ((x_cnt<420)&&(x_cnt>=0)&&(y_cnt>=450)&&(y_cn
t<460)) || ((x_c
        nt<420)&&(x_cnt>=410)&&(y_cnt>=0)&&(y_cnt<460))) rgb<=3'b110;
        else
if(((x_cnt>=0)&&(x_cnt<x_block)&&(y_cnt>=460)&&(y_cnt<480)) if((x_cnt%20)<3
)
        rgb<=3'b000;else rgb<=3'b100;
        else if((x_cnt>=430)&&(x_cnt<630)&&(y_cnt>=0)&&(y_cnt<200))
        begin
        if(x_block<400)
        begin
        if(time1<=20) DOUT1<=y_cnt;else if(time1<=40) DOUT1<=y_cnt+200;else
DOUT1<=y_cnt+400;
        charbit<=629-x_cnt;
        if(rgbin[charbit]) begin rgb<=data%8;if(rgb==3'b011) rgb<=010;end
        else begin rgb<=3'b011;end
        end
        else rgb<=3'b011;
        end
        ////gaile
        else if((x_cnt>=430)&&(x_cnt<630)&&(y_cnt>=200)&&(y_cnt<250))
        begin
        if(x_block==400)
        rgb<=3'b011;
        else
        begin
        DOUT1<=y_cnt+400;
        charbit<=629-x_cnt;
        if(rgbin[charbit]) begin rgb<=data%8;if(rgb==3'b011) rgb<=010;end
        else begin rgb<=3'b011;end
        end
        end

```

```

else if((x_cnt>=100)&&(x_cnt<300)&&(y_cnt>=250)&&(y_cnt<450))
begin
if(x_block==400)
begin
begin if(time1<=20) DOUT1<=400+y_cnt;else if(time1<=40)
DOUT1<=y_cnt+600;else DOUT1<=y_cnt+800;end
charbit<=299-x_cnt;
if(rgbin[charbit]) rgb<=3'b101;else rgb<=3'b011;
end
else
if((x_cnt>=x_ball)&&(x_cnt<x_ball+16)&&(y_cnt>=y_ball)&&(y_cnt<y_bal
1+32))
begin
DOUT<=(y_cnt-y_ball)/2+data*16;a<=q[(x_cnt-x_ball)/2%8];
begin
if((a==1)&&(data1!=data)) begin rgb<=3'b000; aa<=0;end
else if(data1==data) begin aa<=1; rgb<=3'b011 ;end
else begin rgb<=3'b011;aa<=0;end
end
end
else rgb<=3'b011;
end
else if((x_cnt>=100)&&(x_cnt<300)&&(y_cnt>=200)&&(y_cnt<250))
begin
if(x_block==400)
begin
DOUT1<=y_cnt+1050;
charbit<=299-x_cnt;
if(rgbin[charbit]) rgb<=3'b100;else rgb<=3'b011;
end
else
if((x_cnt>=x_ball)&&(x_cnt<x_ball+16)&&(y_cnt>=y_ball)&&(y_cnt<y_bal
1+32))
begin

```

```

DOUT<=(y_cnt-y_ball)/2+data*16;a<=q[(x_cnt-x_ball)/2%8];
begin
if((a==1)&&(data1!=data)) begin rgb<=3'b000; aa<=0;end
else if(data1==data) begin aa<=1; rgb<=3'b011 ;end
else begin rgb<=3'b011;aa<=0;end
end
end
else rgb<=3'b011;
end
else
if((x_cnt>=x_ball)&&(x_cnt<x_ball+16)&&(y_cnt>=y_ball)&&(y_cnt<y_bal
1+32))
if(x_block==400) rgb<=3'b011;
else begin
DOUT<=(y_cnt-y_ball)/2+data*16;a<=q[(x_cnt-x_ball)/2%8];
if((a==1)&&(data1!=data)) begin rgb<=3'b000; aa<=0;end
else if(data1==data) begin aa<=1; rgb<=3'b011 ;end
else begin rgb<=3'b011;aa<=0;end
end
end
always @(posedge clk_25m)
if(rst)
begin
if(aa==1);
end
always @(posedge clk_25m)
begin
if(rst)
begin
case(state)
3'b000: if((y_ball!=10)&&(clk_count)&&(x_ball!=390)) begin
x_ball<=x_ball+1;y_ball<=y_ball-2; flag_x<=1;flag_y<=0; end
else if((y_ball==10)|| (x_ball==390)) begin flag_x<=1;flag_y<=0;end

```

```

3'b001:if((x_ball!=390)&&(clk_count)&&(y_ball!=420)) begin
x_ball<=x_ball+1;y_ball<=y_ball+2; flag_x<=1;flag_y<=1; end
else if((y_ball==420) || ( x_ball==390)) begin flag_x<=1;flag_y<=1;end
3'b010:if((y_ball!=420)&&(clk_count)&&(x_ball!=10)) begin
x_ball<=x_ball-1;y_ball<=y_ball+2; flag_x<=1;flag_y<=0; end
else if((y_ball==420) || ( x_ball==10)) begin flag_x<=0;flag_y<=1;end
3'b011:if((y_ball!=10)&&(x_ball!=10)&&(clk_count)) begin
x_ball<=x_ball-1;y_ball<=y_ball-2; flag_x<=0;flag_y<=0; end
else if((y_ball==10) || ( x_ball==10)) begin flag_x<=0;flag_y<=0;end
3'b100:if((y_ball!=420)&&(clk_count)&&(x_ball!=390)) begin
x_ball<=x_ball+1;y_ball<=y_ball+2;flag_x<=1;flag_y<=1; end
else if((y_ball==420) || ( x_ball==390)) begin flag_x<=1;flag_y<=1;end
3'b101:if((y_ball!=10)&&(clk_count)&&(x_ball!=390)) begin
x_ball<=x_ball+1;y_ball<=y_ball-2; flag_x<=1;flag_y<=0; end
else if((y_ball==10) || (x_ball==390)) begin flag_x<=1;flag_y<=0;end
3'b110:if((y_ball!=10)&&(clk_count)&&(x_ball!=10)) begin
x_ball<=x_ball-1;y_ball<=y_ball-2; flag_x<=0;flag_y<=0; end
else if((y_ball==10) || ( x_ball==10)) begin flag_x<=0;flag_y<=0;end
3'b111:if((y_ball!=420)&&(clk_count)&&(x_ball!=10)) begin
x_ball<=x_ball-1;y_ball<=y_ball+2; flag_x<=0;flag_y<=1; end
else if((y_ball==420) || (x_ball==10)) begin flag_x<=0;flag_y<=1;end
endcase
end
else
begin
x_ball<=240;
y_ball<=120;
end
end
always @ (posedge clk_25m)
if(rst)
begin
if ((flag_x==0)&&(flag_y==0)&&(x_ball==10)) state<=3'b000;
else if((flag_x==1)&&(flag_y==0) &&(y_ball==10)) state<=3'b001;

```

```

else if((flag_x==1)&&(flag_y==1)&&(x_ball==390)) state<=3'b010;
else if((flag_x==0)&&(flag_y==1)&&(y_ball==420)) state<=3'b011;
else if((flag_x==0)&&(flag_y==1)&&(x_ball==10)) state<=3'b100;
else if((flag_x==1)&&(flag_y==1)&&(y_ball==420)) state<=3'b101;
else if((flag_x==1)&&(flag_y==0)&&(x_ball==390)) state<=3'b110;
else if ((flag_x==0)&&(flag_y==0)&&(y_ball==10)) state<=3'b111;
else state<=state;
end
else state<=3'b011;
endmodule

```

4、实验仿真：

根据描述语言进行仿真测试，分析波形。

5、管脚约束：

```

set_pin_assignment { b } { LOCATION = E13; IOSTANDARD = LVCMOS33; DRIVESTRENGTH = 8;
PULLTYPE = NONE; }
set_pin_assignment { clk } { LOCATION = R7; IOSTANDARD = LVCMOS33; PULLTYPE = PULLUP; }
set_pin_assignment { g } { LOCATION = N6; IOSTANDARD = LVCMOS33; DRIVESTRENGTH = 8;
PULLTYPE = NONE; }
set_pin_assignment { hs } { LOCATION = E7; IOSTANDARD = LVCMOS33; DRIVESTRENGTH = 8;
PULLTYPE = NONE; }
set_pin_assignment { ps2_clk } { LOCATION = M16; IOSTANDARD = LVCMOS33; PULLTYPE =
PULLUP; }
set_pin_assignment { ps2_dat } { LOCATION = K16; IOSTANDARD = LVCMOS33; PULLTYPE =
PULLUP; }
set_pin_assignment { r } { LOCATION = D5; IOSTANDARD = LVCMOS33; DRIVESTRENGTH = 8;
PULLTYPE = NONE; }
set_pin_assignment { rst } { LOCATION = A2; IOSTANDARD = LVCMOS33; PULLTYPE = PULLUP; }
set_pin_assignment { spk_kk } { LOCATION = H11; IOSTANDARD = LVCMOS33; DRIVESTRENGTH
= 8; PULLTYPE = NONE; }
set_pin_assignment { vs } { LOCATION = F7; IOSTANDARD = LVCMOS33; DRIVESTRENGTH = 8;
PULLTYPE = NONE; }

```

6、下载测试：

将 PS/2 键盘链接扩展模块 PS/2 接口， VGA 连接开发板接口。

将连接线连接至学习板的 USB 下载 端口，并连接计算机的 USB 接口后，给板上电。进

行程序下载后，程序下载完毕。KEY1 复位，显示屏跳出字符，根据字符敲打键盘对应字符，看屏幕出现什么效果。

